SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

Dr Geetha

QUESTIONS: 10 Total Marks:50

Annexure A

Scenario based

Code of Conduct:

I **Galla Sai Siddartha (192425093)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : **Galla Sai Siddartha**

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long. Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify *why* your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

ANSWER

1. Understanding of the Scenario

The booking server handles three kinds of work at the same time during festival seasons. Payment transactions (P1) are the most time-critical because a delayed payment can time out and cancel a booking. Seat availability checks (P2) must be answered quickly so that users see correct seat data. Ticket confirmation and printing (P3) is the least urgent step and has the longest bursts, up to 20 ms. Because the CPU is shared by all three, the order in which the scheduler picks processes decides how long every customer waits.

2. Identification of Key Issues

- The current order of execution does not reflect how important each process is, so short, urgent jobs may wait behind a long printing job.
- Heavy festival load creates long ready queues, which raises waiting time and turnaround time.

• Confirmation (P3) is slow mainly because it is stuck in the same queue as other tasks and is served in arrival order, so the whole booking looks delayed. Burst times also differ widely (2–20 ms), causing the convoy effect.

3. Application of Concepts and Chosen Algorithm

The most suitable method is Priority Scheduling (preemptive, with aging). FCFS ignores urgency, SJF needs burst prediction and can starve long jobs, and Round Robin gives equal importance to payment and printing and adds context-switch overhead. Priority scheduling maps directly to the three categories: P1 gets priority 1, P2 priority 2 and P3 priority 3. Within the same priority level, processes are served in FCFS or Round Robin order. Aging gradually raises the priority of a process that has waited for a long time, so P3 is never starved.

4. Decision Making and Justification

Priority scheduling improves CPU utilization because the CPU always has a ready process to run and the most valuable work is completed first, so payments finish quickly and release their resources (database connections, locks and memory) early. It reduces customer wait time because short, urgent processes no longer queue behind long ones. In the sample below, the average waiting time falls from 16.67 ms under FCFS (arrival order P3, P2, P1) to 7.33 ms under priority scheduling, and the average turnaround time falls from 28.67 ms to 19.33 ms. Aging then ensures that confirmation and printing still complete within a bounded time, which answers the original complaint.

5. Sample Execution Order

Assume P1 = 6 ms, P2 = 10 ms and P3 = 20 ms, all arriving at time 0. The Gantt chart is: | P1 (0–6) | P2 (6–16) | P3 (16–36) |.

Process	Burst (ms)	Priority	Waiting (ms)	Turnaround (ms)
P1 – Payment	6	1	0	6
P2 – Seat check	10	2	6	16
P3 – Confirmation	20	3	16	36
Average	—	—	7.33	19.33

6. Python Program

```
# Priority scheduling vs FCFS (all processes arrive at t = 0)
procs = [("P3", 20, 3), ("P2", 10, 2), ("P1", 6, 1)]  # name, burst, priority

def run(order, title):
    t, wait, tat, gantt = 0, 0, 0, ""
    print(f"--- {title} ---")
    print(f"{'Proc':<5}{'Burst':<7}{'Start':<7}{'End':<6}{'Wait':<6}{'TAT'}")
    for name, burst, _ in order:
        start, end = t, t + burst
        print(f"{name:<5}{burst:<7}{start:<7}{end:<6}{start:<6}{end}")
        wait, tat, t = wait + start, tat + end, end
        gantt += f"| {name} ({start}-{end}) "
    print("Gantt :", gantt + "|")
    print(f"Average waiting time    = {wait / len(order):.2f} ms")
    print(f"Average turnaround time = {tat / len(order):.2f} ms")
    print("CPU utilization         = 100 %\n")

run(procs, "FCFS (arrival order P3, P2, P1)")
run(sorted(procs, key=lambda p: p[2]), "Priority scheduling (P1, P2, P3)")
```

7. Output

```
--- FCFS (arrival order P3, P2, P1) ---
Proc Burst  Start  End   Wait  TAT
P3   20     0      20    0     20
P2   10     20     30    20    30
P1   6      30     36    30    36
Gantt : | P3 (0-20) | P2 (20-30) | P1 (30-36) |
Average waiting time    = 16.67 ms
Average turnaround time = 28.67 ms
CPU utilization         = 100 %

--- Priority scheduling (P1, P2, P3) ---
Proc Burst  Start  End   Wait  TAT
P1   6      0      6     0     6
```

P2 10 6 16 6 16
P3 20 16 36 16 36
Gantt : | P1 (0-6) | P2 (6-16) | P3 (16-36) |
Average waiting time = 7.33 ms
Average turnaround time = 19.33 ms
CPU utilization = 100 %

8. Drawback in Real-Time Systems

The main drawback of priority scheduling is starvation (indefinite blocking): if high-priority payment and seat-check requests keep arriving during festival peaks, low-priority confirmation and printing may wait for a very long time or never run. In a real-time system this also causes priority inversion when a low-priority process holds a resource needed by a high-priority one. Aging and priority inheritance reduce both problems, so preemptive priority scheduling with aging remains the best overall choice here.

RUBRICS FOR CALCULATION

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding ; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand